Natural Language Processing
11. BERT & GPT

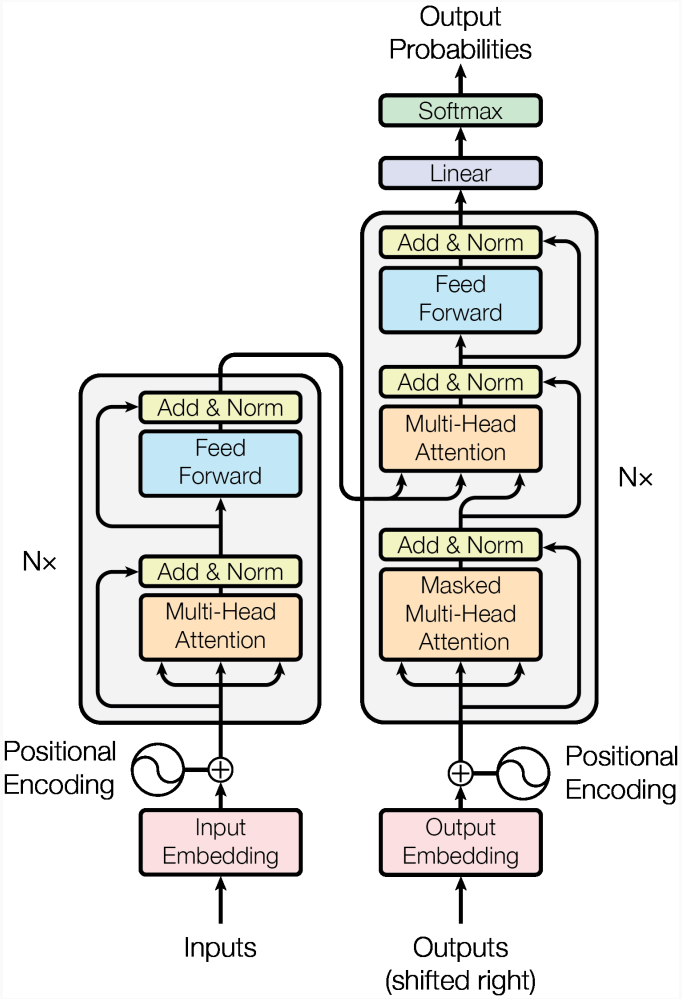
Juan José Alegría

June 3, 2025

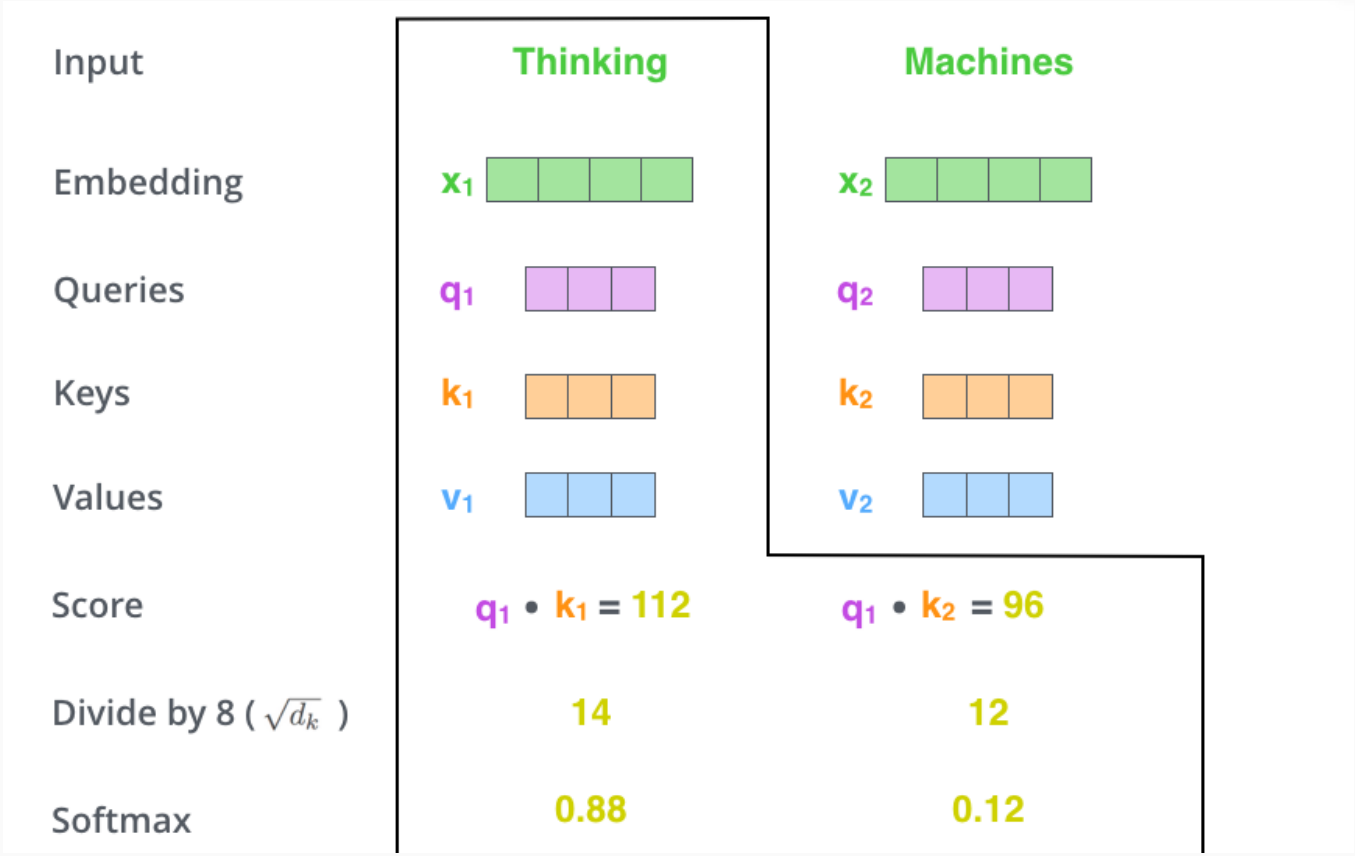
Transformers Recap

Transformers Recap

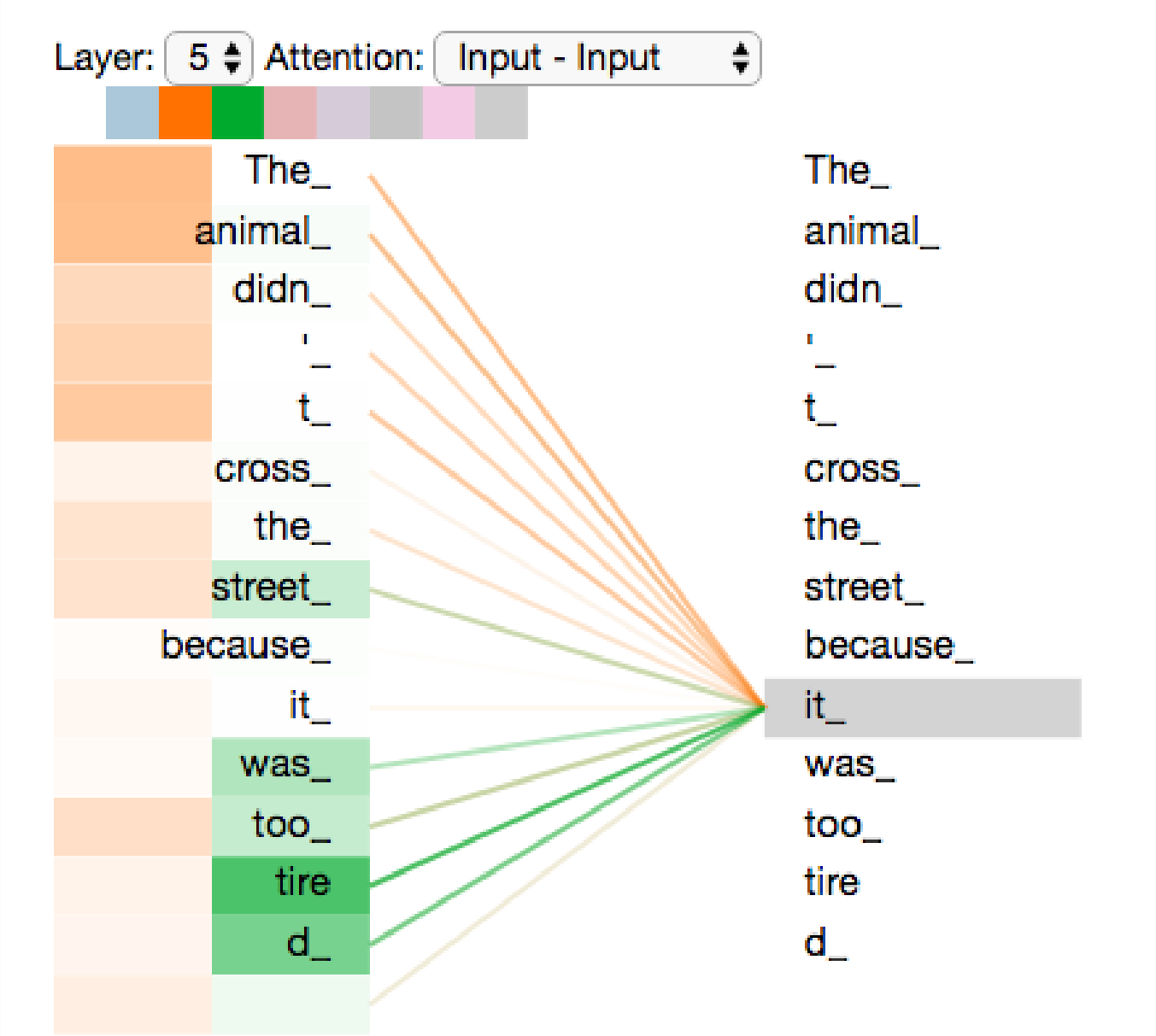
- Introduced in the paper *Attention is All You Need* by researchers at Google [Vaswani et al., 2017].
- Originally designed to address the sequence-to-sequence (seq2seq) problem.
- Consists of both an encoder and a decoder.
- Key innovation: replaces recurrence entirely with *attention* mechanisms — within the encoder, within the decoder, and between them.
- The encoder attends to the entire input sequence, while the decoder attends only to previously generated tokens (via masking).



Self-attention



Self-attention



BERT

BERT

- Introduced in the paper *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding* by Google Researchers [Devlin et al., 2019].
- **BERT**: **B**idirectional **E**ncoder **R**epresentations from **T**ransformers.
- Core idea: create **bidirectional** representations from text; i.e., each word can look both to its left and its right.
- Core idea: learn bidirectional contextual representations, where each word is informed by both its left and right context.
- Architecturally, BERT consists of a stack of Transformer **encoders** only.
- Extensively pre-trained on raw, unlabeled textual data using a self-supervised learning objective.
- Then it's fine-tuned to solve downstream tasks (e.g., question answering, sentence classification, named entity recognition, etc).

BERT

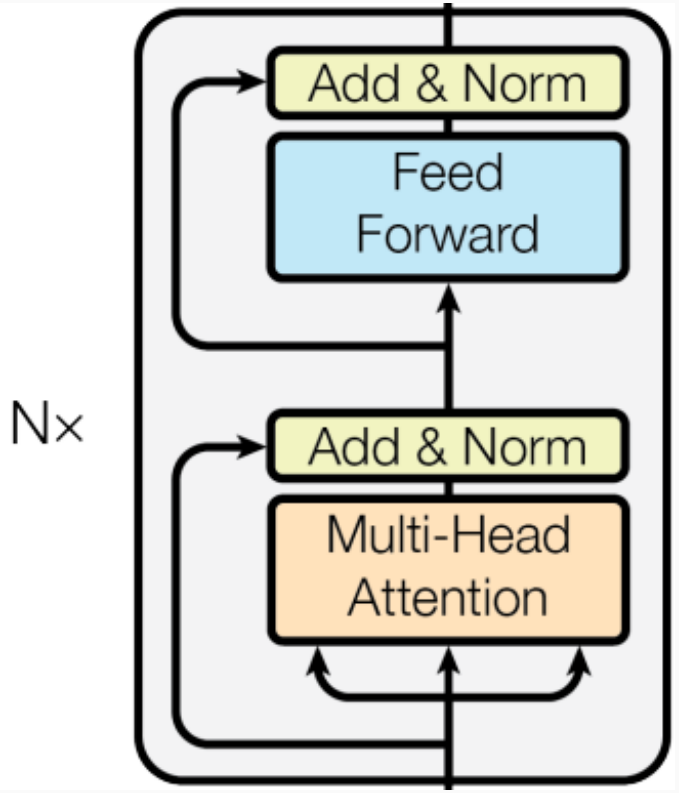
- Prior to BERT, language models typically used unidirectional self-attention or shallow bidirectional approaches that combined left-to-right and right-to-left representations.
- BERT introduced a new paradigm by predicting masked tokens in the middle of a sentence, leveraging both left and right context.
- This approach results in rich, deeply contextualized representations.
- However, it also means that BERT is not inherently suited for generative tasks—i.e., it cannot predict the next token in a sequence. That said, some research explores using two BERT models (one as an encoder and one as a decoder) for sequence-to-sequence tasks. For more information, see the Hugging Face documentation on BERT generation and [Rothe et al., 2020].

BERT: use cases

- Although BERT cannot be directly used for text generation, it is highly effective for a wide range of natural language understanding tasks, such as sentence classification, textual entailment (predicting whether one sentence logically follows from another), question answering (identifying the span in a passage that answers a given question), part-of-speech (POS) tagging, named entity recognition (NER), and more.
- Another major use case of BERT is generating contextualized embeddings for words, sentences, or even entire paragraphs. These embeddings can be used for tasks such as semantic search—for example, finding the most similar document to a given query based on embedding similarity.

BERT architecture

- As previously mentioned, the architecture of BERT is just the encoder portion of a transformer model.
- In the original paper, two model sizes were introduced:
  - **BERT Base**: 12 layers (transformer blocks), 768-dimensional hidden representations, 12 attention heads per self-attention layer, $\approx$ 110 million parameters.
  - **BERT Large**: 24 layers, 1024-dimensional hidden representations, 16 attention heads per layer, $\approx$ 340 million parameters.
- Notably, BERT Base was comparable in size to the first version of OpenAI's GPT model [Radford et al., 2018].



BERT: bidirectional self-attention

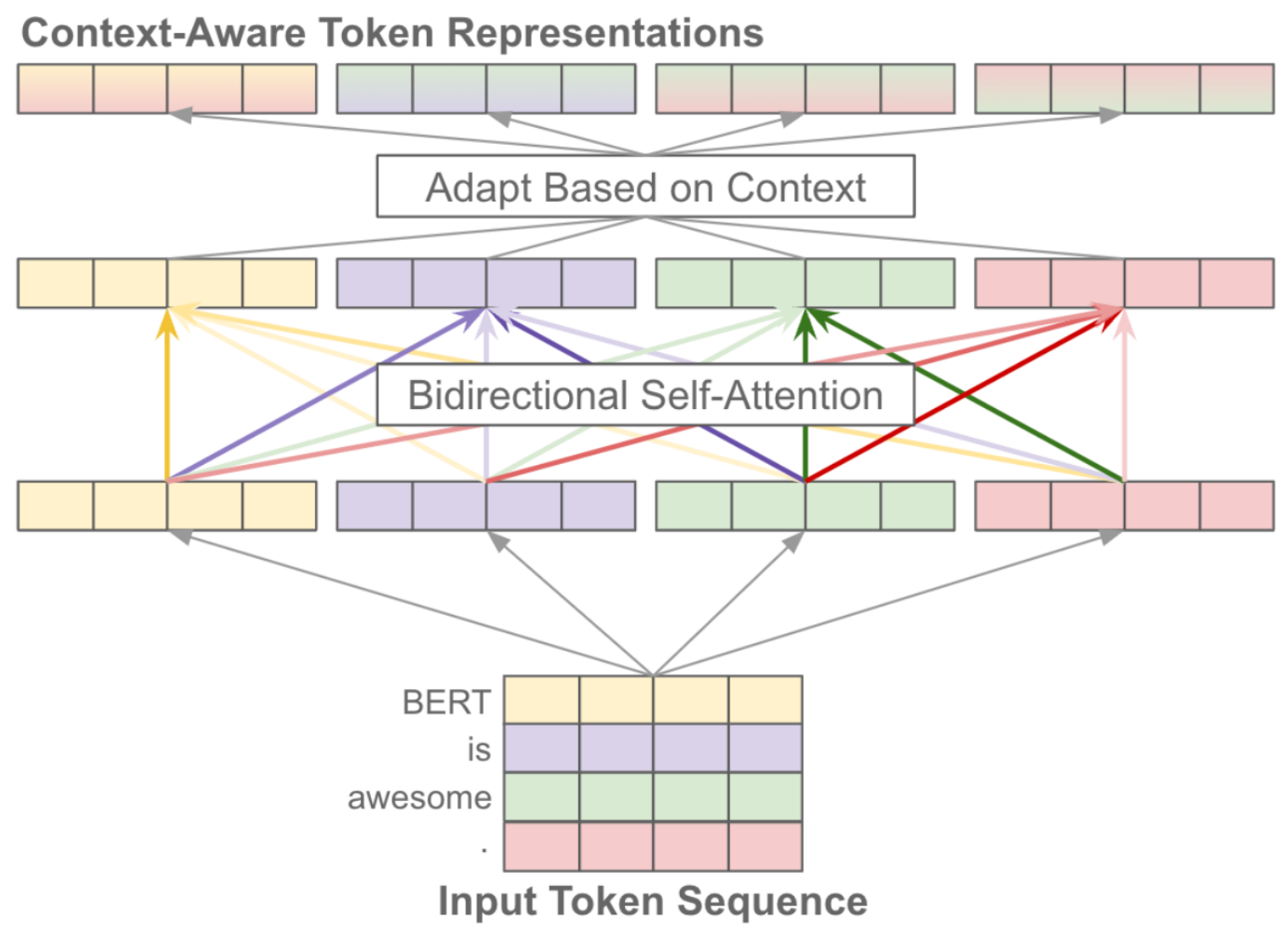


Figure 1: Self-attention adapts each token's vector representation based on the other tokens within the sequence, forming a more context-aware representation.

BERT: bidirectional self-attention

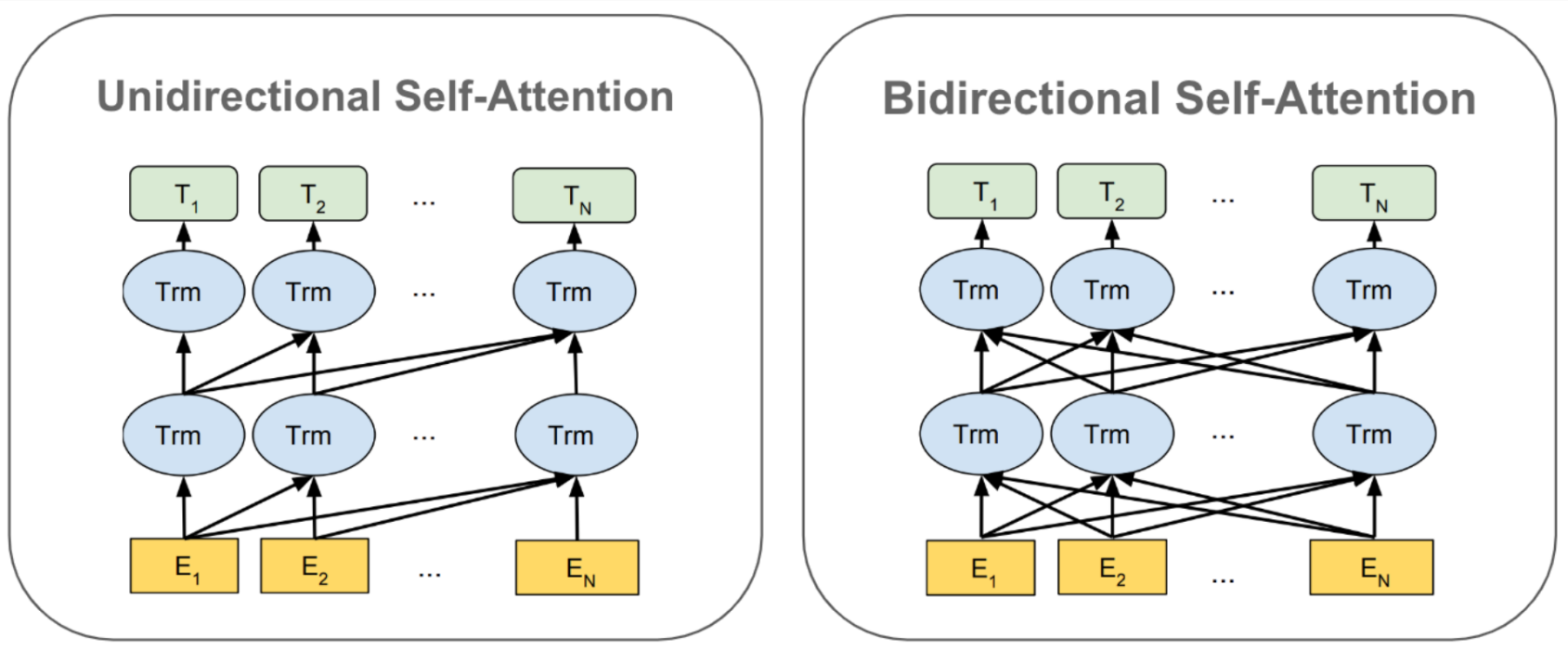


Figure 2: Unidirectional vs bidirectional self-attention.

BERT Input

- BERT input is defined as a single *sequence*.
- A *sequence* can consist of either one *sentence* or two *sentences* packed together.
- A *sentence* is defined as an “arbitrary span of contiguous text” (not necessarily a grammatical sentence).
- The entire sequence is first tokenized.
- A special `[CLS]` token is always inserted at the beginning of the sequence.
- Each sentence is terminated with a `[SEP]` token. Thus, there will be one or two `[SEP]` tokens depending on whether the input consists of one or two sentences.

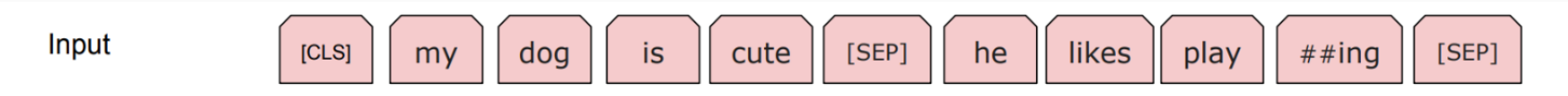


Figure 3: BERT tokenized input

BERT Input Embeddings

- Each token is mapped to a corresponding embedding vector using a learned embedding matrix (a lookup table).
- The input is thus represented as a sequence of vectors, one per token.
- At each position, two additional vectors are added: one encodes the sentence segment (first or second sequence), and the other encodes the token's position in the sequence.
- These segment and position embeddings are both **learned**, in contrast to the original Transformer implementation [Vaswani et al., 2017], which used fixed sine and cosine positional encodings.

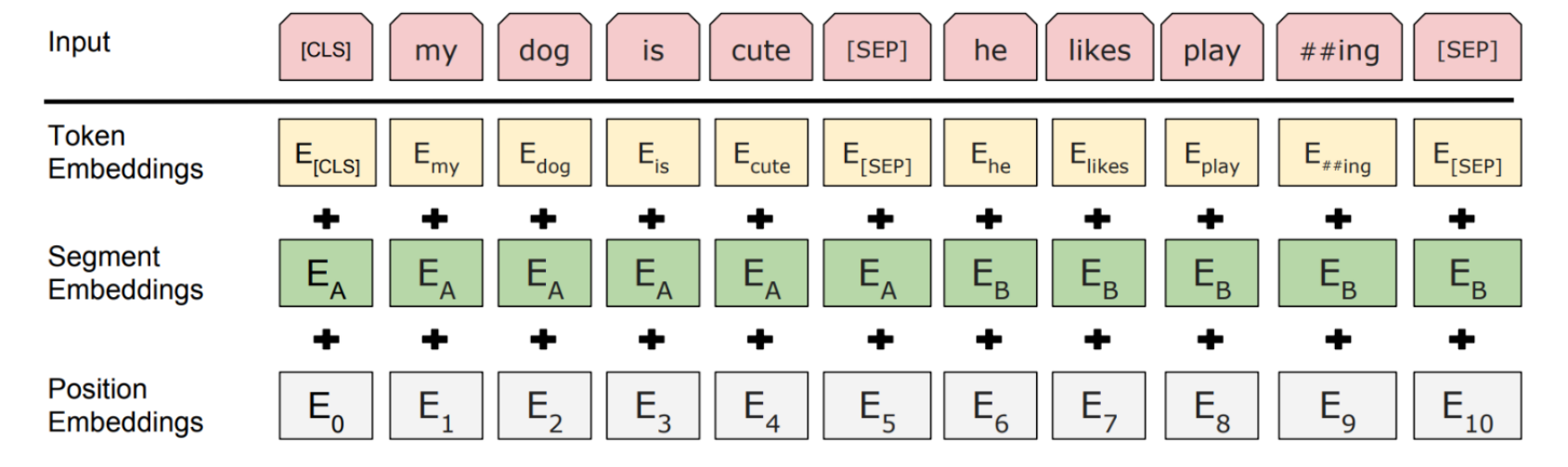


Figure 4: BERT tokenized input

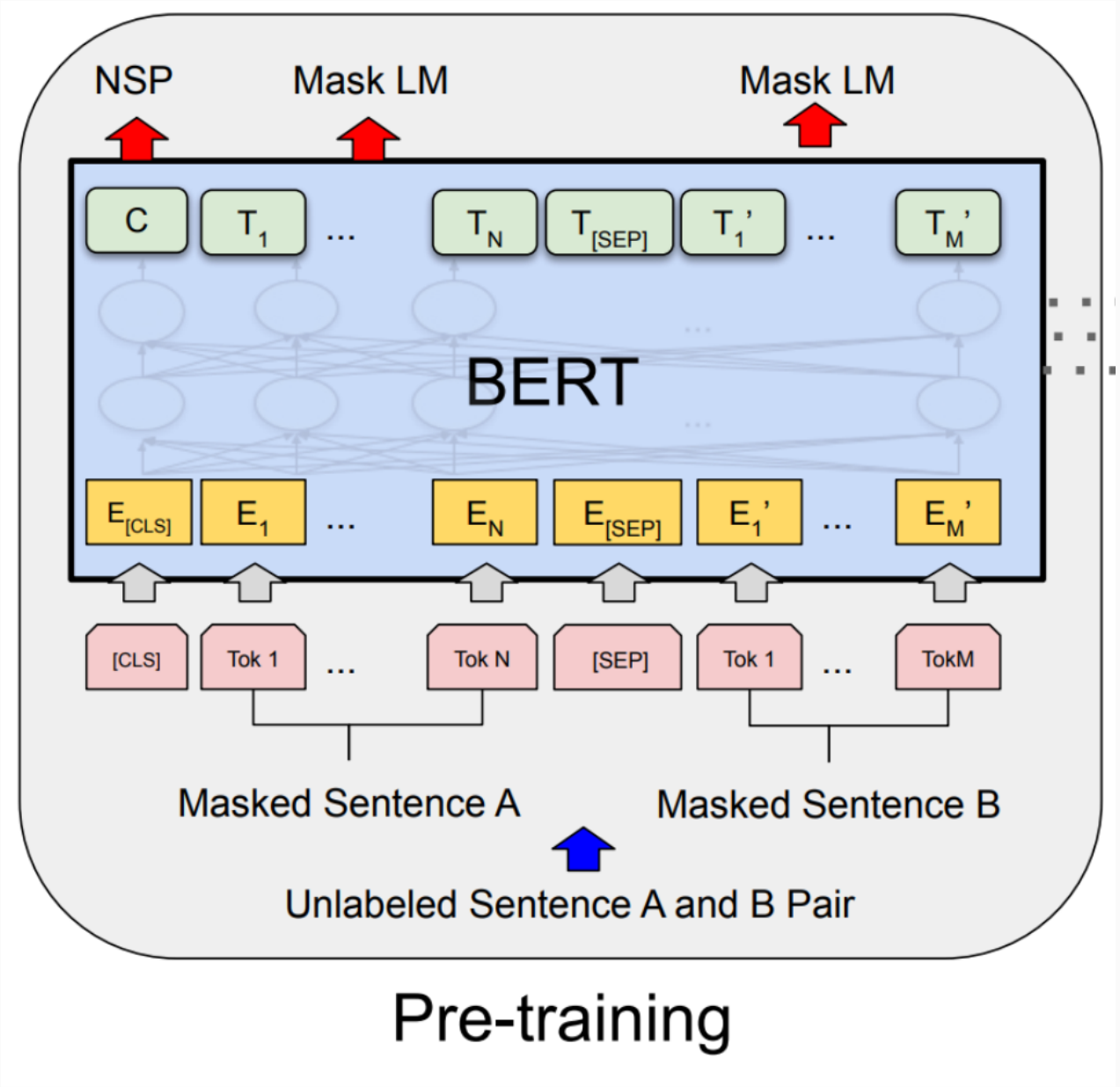
BERT training

- The training process for BERT proceeds in two steps:
  - Pre-training
  - Fine-tuning
- The architecture is nearly identical between these steps, though some small, task-specific modules may be used.

Pre-training

- BERT is pre-trained in a **self-supervised** manner: it uses unlabeled raw text, but leverages the structure of the text to create a learning signal. **Note:** This differs from unsupervised learning, where no explicit supervision signal is provided.
- The model is trained on two pretraining tasks:
    - **Masked Language Modeling (MLM)**: Randomly masks some tokens in the input and trains the model to predict them based on surrounding context.
    - **Next Sentence Prediction (NSP)**: Given a pair of sentences, the model learns to predict whether the second sentence logically follows the first in the original corpus.

Pre-training



Pre-training

Pre-training: Next Sentence Prediction

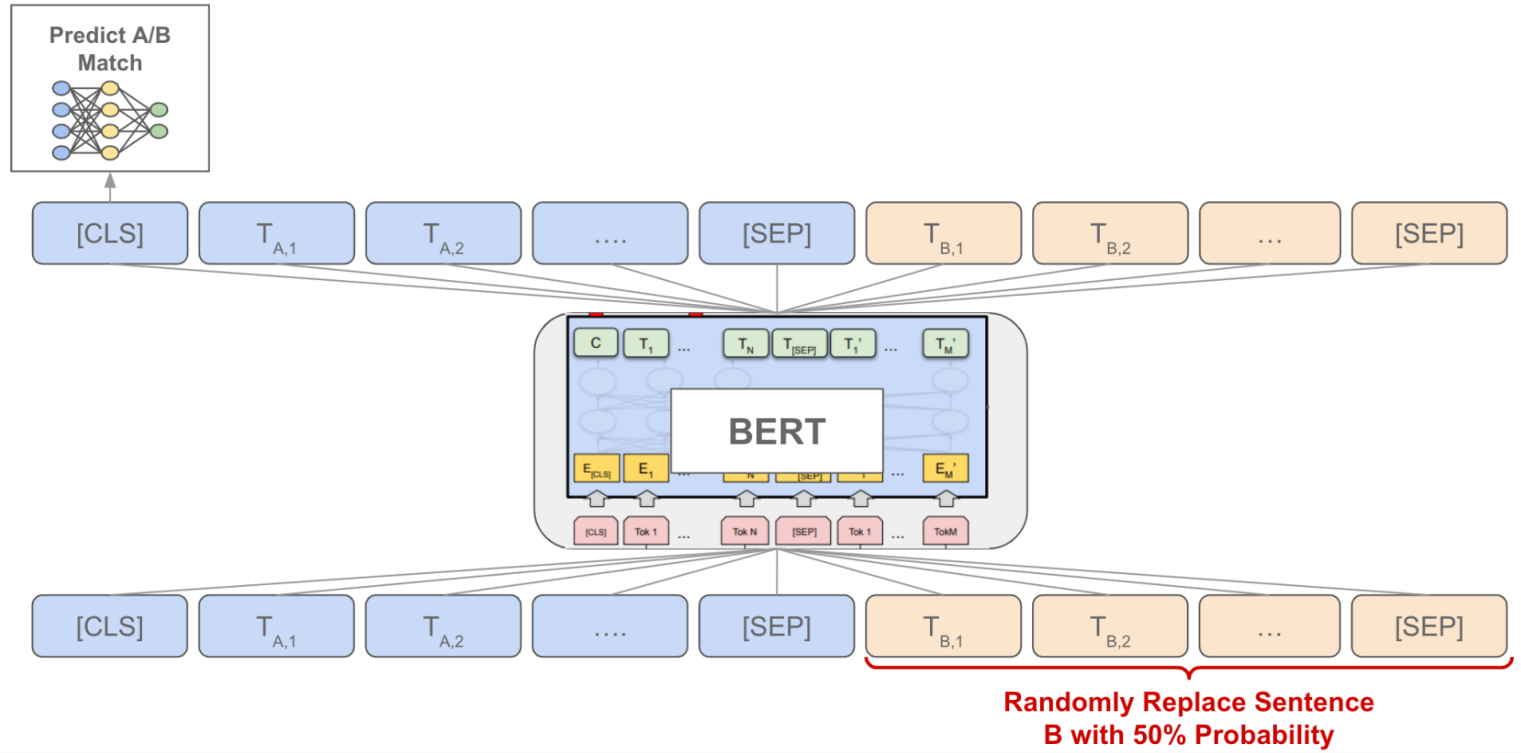


Figure 5: Consecutive sentences from the corpus (Sentence A and Sentence B) are passed into BERT. In 50% of the cases, Sentence B is replaced with a random sentence from the corpus. The final representation of the `[CLS]` token is then passed through a classification layer to predict whether Sentence B logically follows Sentence A. In other words, it is formulated as a binary classification task.

Pre-training: Masked Language Modeling

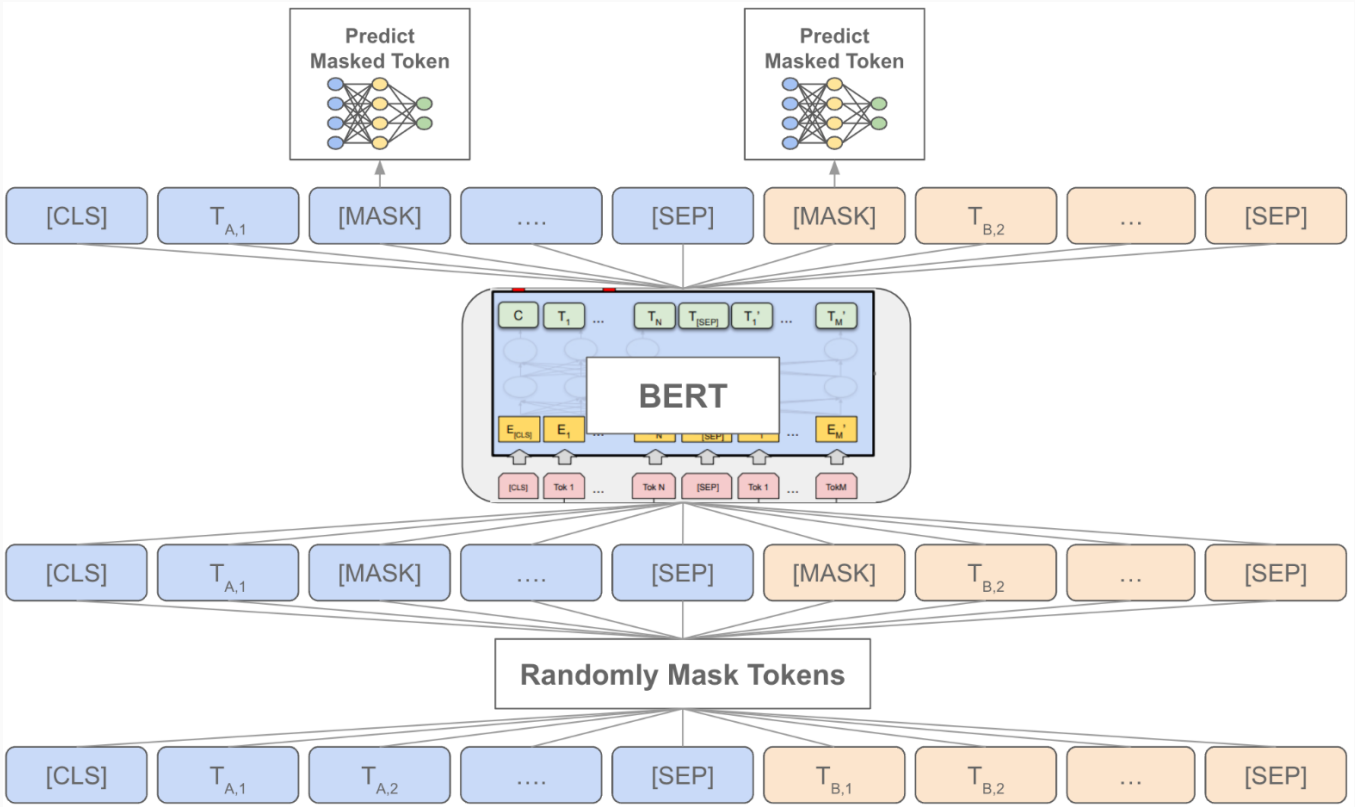


Figure 6: First, 15% of the tokens in the input sequence are masked and replaced with the special `[MASK]` token. The final hidden representations of these `[MASK]` tokens are then passed through a classification layer to predict the original tokens that were masked. The objective is to recover the original masked words.

Pre-training: details

- Note that BERT cannot be trained using the typical language modeling objective of predicting the next word in a sequence. Due to its bidirectional self-attention mechanism, BERT could simply ”cheat” by attending to the token it is supposed to predict.
- Not all masked tokens are replaced with `[MASK]`. Of the 15% of tokens selected for masking:
  - 80% are replaced with the `[MASK]` token,
  - 10% are replaced with a random token,
  - and 10% are left unchanged.

  This design helps mitigate the mismatch between pre-training (which uses `[MASK]`) and fine-tuning (where `[MASK]` does not appear). It also encourages the model to: (i) predict a token given its context even when the token is visible, and (ii) identify and correct incorrect tokens.

Pre-training: details

- BERT is pre-trained on the two tasks (NSP and MLM) **simultaneously**.
- For each training example, two sentences are packed together as input. Then, some tokens in the sequence are randomly selected for masking.
- The full input is passed through BERT, which produces:
  - A representation T_i for each input token Tok_i.
  - A special representation C for the `[CLS]` token.
- The `[CLS]` representation C is passed through a binary classification head to predict whether sentence B follows sentence A. This yields the NSP loss: loss_{nsp}.
- The representations T_i corresponding to the masked tokens are passed through another classification head to predict the original masked words. This gives the MLM loss: loss_{mlm}.
- The total training loss is the sum of both objectives: $\text{loss}_{\text{total}} = \text{loss}_{\text{nsp}} + \text{loss}_{\text{mlm}}$.

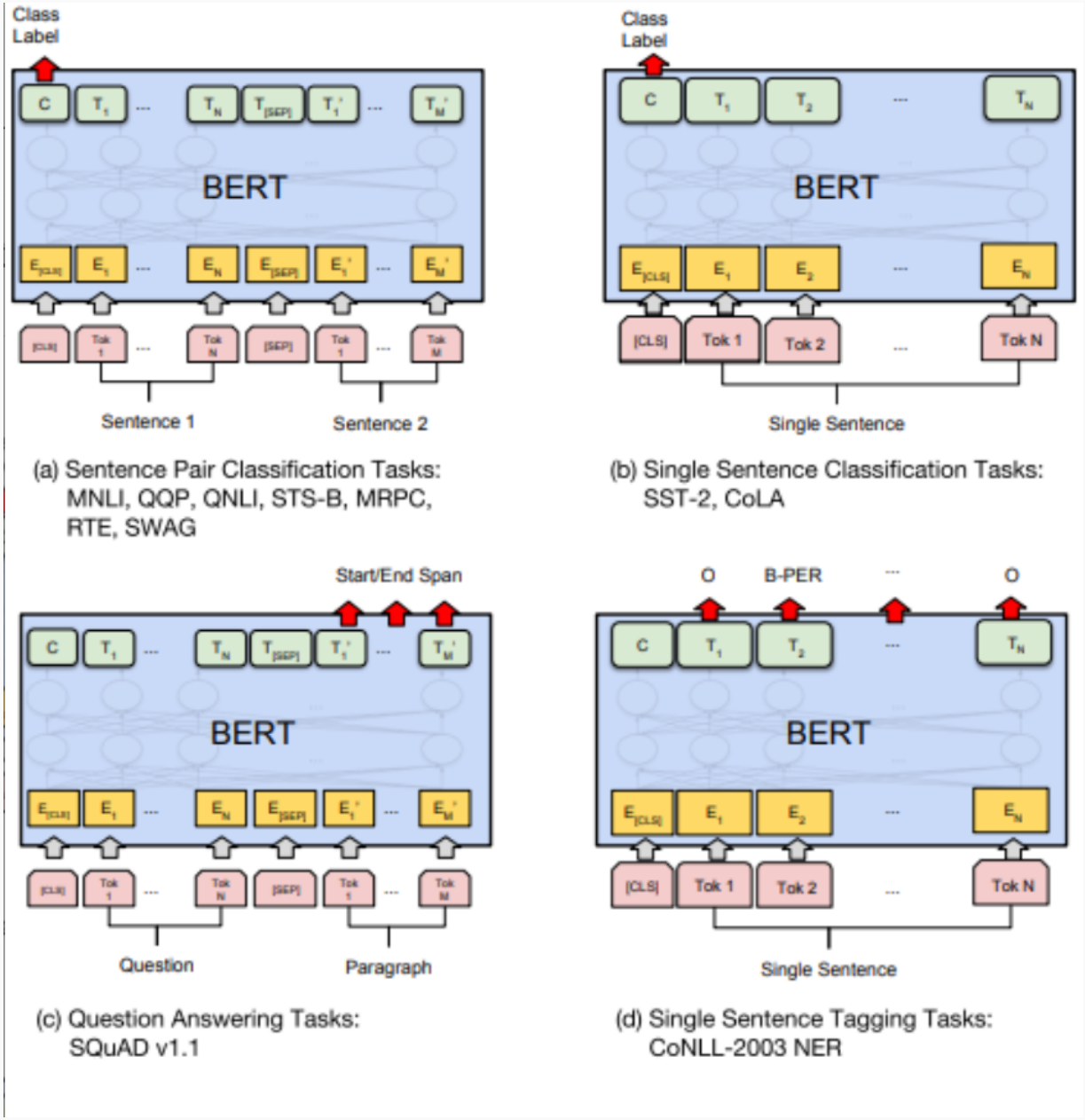
Fine-tuning

- BERT is designed so that adapting it to a variety of downstream tasks is straightforward. You only need to match the input-output structure of the task to that of BERT, add an appropriate task-specific head on top, and fine-tune all model parameters.
- Common task types:
  - **Token-level tasks**: Process the input sequence as usual, then pass each token's output representation through a classification head to make token-level predictions. *Examples:* Named Entity Recognition (NER), Part-of-Speech (POS) tagging.
  - **Single-sentence classification tasks**: Process the sequence normally, then use the output of the `[CLS]` token (which acts as an aggregate embedding) for classification. *Examples:* Sentiment analysis, grammatical acceptability classification.

Fine-tuning

- Common task types:
  - **Text pair classification tasks**: Encode each part of the pair as “sentence A” and “sentence B” using BERT’s input format. Use the output of the `[CLS]` token to perform classification over the pair. *Examples:* Natural Language Inference (NLI), semantic textual similarity.
  - **Question answering tasks**: Encode the question as sentence A and the context paragraph as sentence B. Then, use the output token representations to predict the start and end positions of the answer span within the paragraph.

Fine-tuning


Class Label
BERT
Sentence 1
Sentence 2
(a) Sentence Pair Classification Tasks: MNLI, QQP, QNLI, STS-B, MRPC, RTE, SWAG
Class Label
BERT
Single Sentence
(b) Single Sentence Classification Tasks: SST-2, CoLA
Start/End Span
BERT
Question
Paragraph
(c) Question Answering Tasks: SQuAD v1.1
O
B-PER
O
BERT
Single Sentence
(d) Single Sentence Tagging Tasks: CoNLL-2003 NER

Fine-tuning

- The core idea of fine-tuning is that BERT has already learned a great deal about language during pre-training. To solve a specific task, you just need to: i) add a task-specific head on top of BERT, and ii) make small adjustments to the pre-trained weights.
- In most cases, the only weights trained from scratch are those in the added head. As a result, fine-tuning is relatively fast. According to the original paper, BERT can be fine-tuned in just a few hours on a single GPU.
- The heavy lifting —pre-training— is already done. You only need to adapt BERT to your task!
- With this approach (pre-training + fine-tuning), BERT achieved state-of-the-art performance on 11 NLP tasks!

Beyond BERT

- Since BERT's release, many improved or specialized variants have been proposed. Some notable examples include:
  - **RoBERTa** – more robust training procedure (no NSP, more data, longer training)
  - **DistilBERT** – lighter and faster using knowledge distillation
  - **ALBERT** – smaller model via parameter sharing and factorized embeddings
  - **XLM-RoBERTa** – multilingual model trained on 100+ languages
  - **BETO** – Spanish BERT model trained by researchers at DCC, Universidad de Chile
- All these models introduce innovations to make BERT faster, more efficient, multilingual, or more accurate—but the core architecture remains largely the same.

Resources to Understand BERT

- **[Devlin et al., 2019]** – The original BERT paper.
- **Language Understanding with BERT** – A detailed blog post explaining BERT's architecture and training objectives.
- **StatQuest – Encoder-Only Transformers (like BERT) Clearly Explained!** – A beginner-friendly YouTube video introducing encoder-only Transformers.
- **[Phuong and Hutter, 2022]** – Paper that formalizes many Transformer algorithms, including those used in encoder-only models like BERT.
- **BERT-pytorch (GitHub)** – PyTorch implementation of BERT.

GPT

GPT

- Introduced in the paper *Improving Language Understanding by Generative Pre-Training* [Radford et al., 2018].
- GPT was released **before** BERT.
- **GPT** stands for **G**enerative **P**re-trained **T**ransformer.
- It is trained with a causal language modeling objective: predict the next token given the previous ones.
- It uses **masked** (causal) self-attention: each token can only attend to previous tokens in the sequence.
- Its architecture is based on the **decoder** part of the original Transformer (without cross-attention between the encoder and decoder, because there is no encoder in this architecture).

Masked self-attention

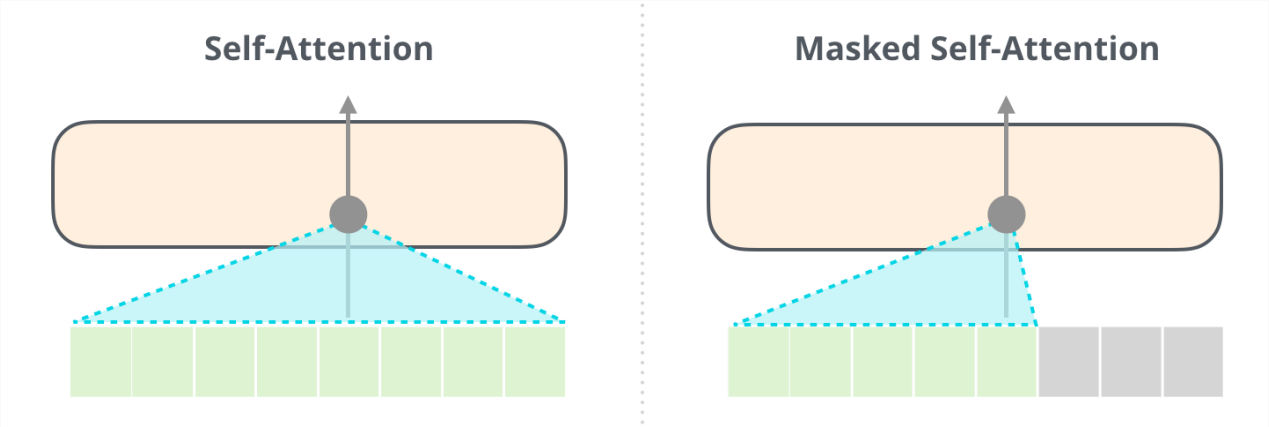


Figure 7: In standard (bidirectional) self-attention, each token can attend to both the tokens to its left and to its right. In contrast, in masked (or causal) self-attention, each token can attend only to the tokens to its left (i.e., preceding tokens).

Masked self-attention

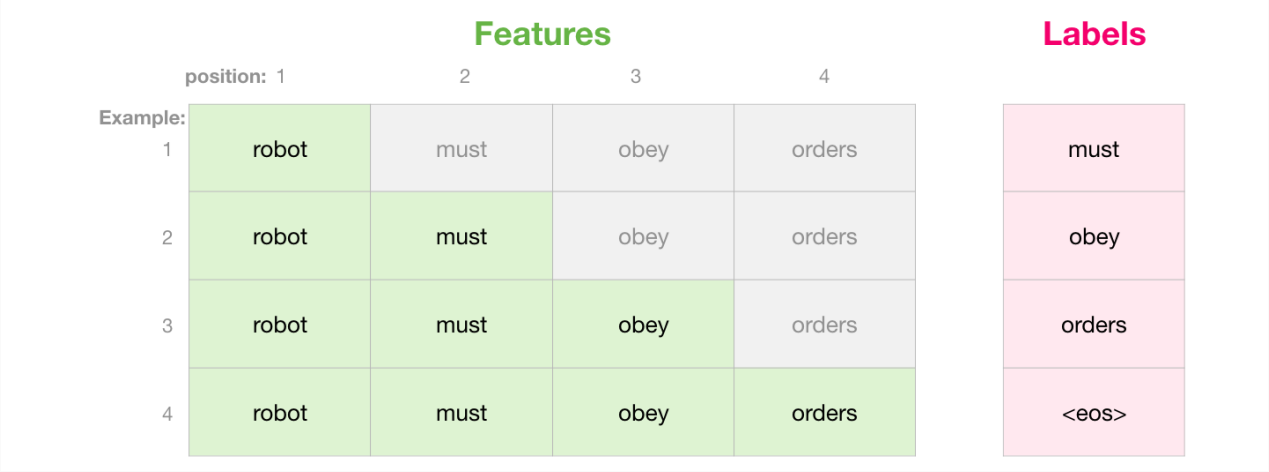


Figure 8: Remember that during training, the model is fed the entire sentence (e.g., `robots must obey orders <eos>`). For each token at position i, the objective is to predict the token at position $i + 1$. Since the full sentence is available during training, we must prevent the model at position i from looking ahead to $i + 1$ and beyond—otherwise, it could simply copy the next token instead of learning meaningful dependencies

Masked self-attention

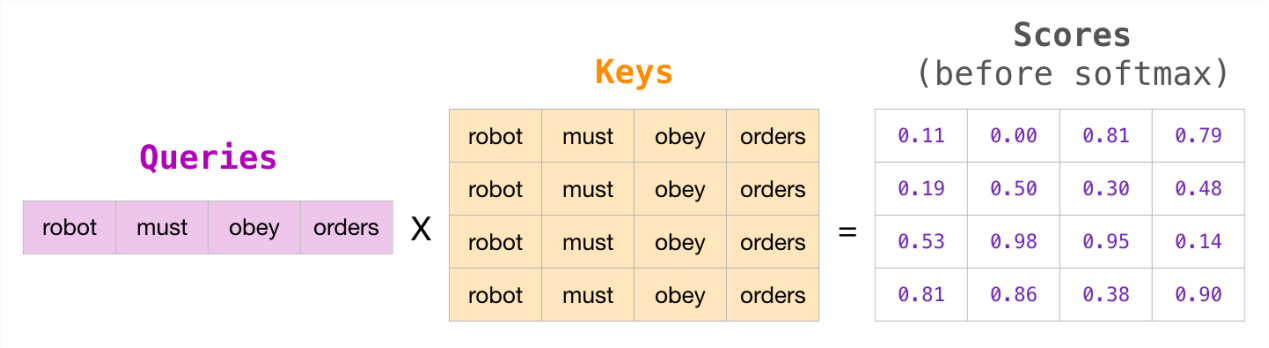


Figure 9: To implement masked self-attention, we first compute the scores between the *queries* and *keys* in the usual way: by taking the dot product of each query with each key.

Masked self-attention

Figure 10: Then, we apply an attention mask by setting all values above the diagonal to $-\infty$ (or a very large negative number).

Masked self-attention

Figure 11: Finally, we compute the softmax over the masked attention scores.

Decoder-only block

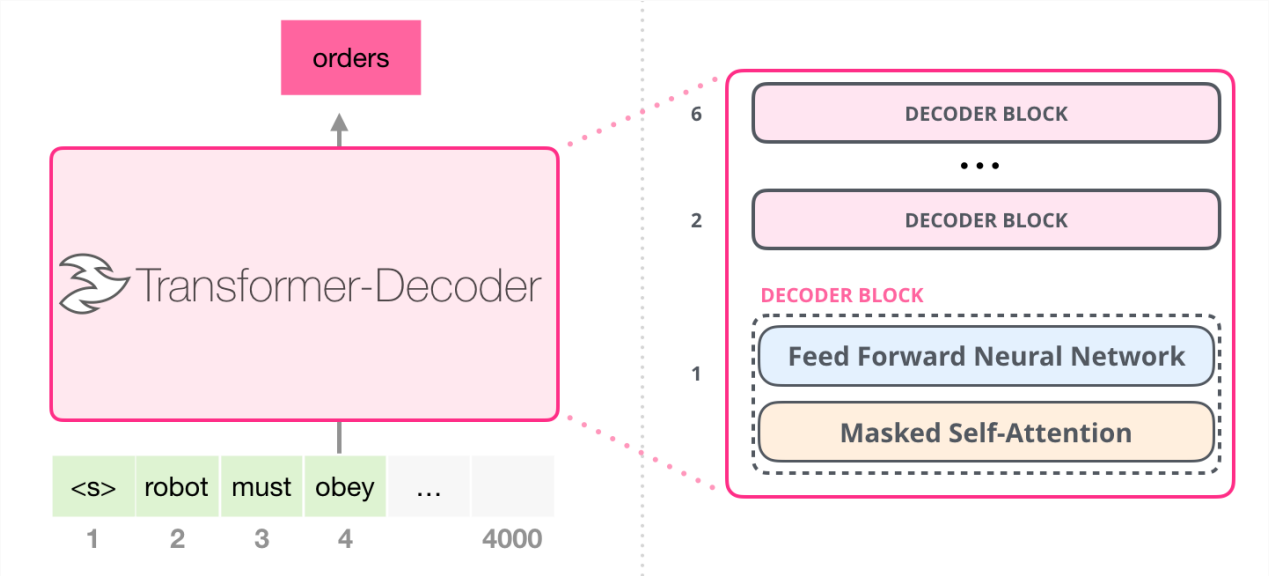


Figure 12: The model is formed by decoder blocks, each formed by self-attention and a feed-forward network.

Output

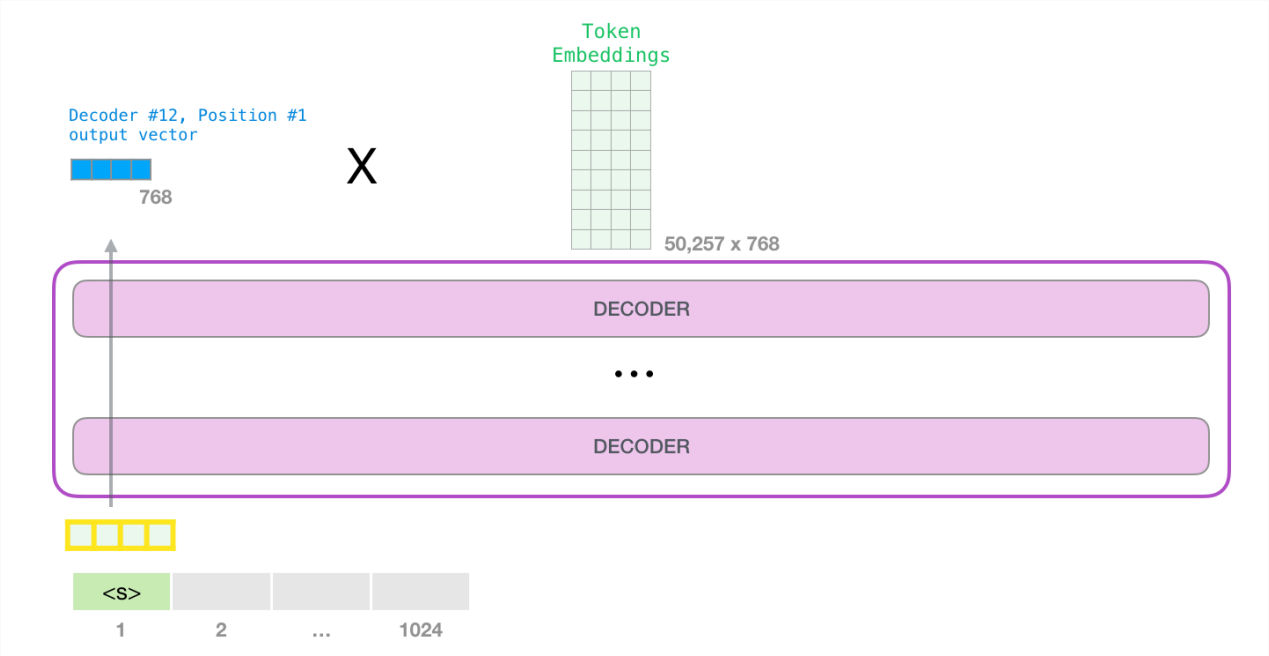


Figure 13: When the final decoder block generates its output, the model multiplies that vector by the transposed embedding matrix.

Output

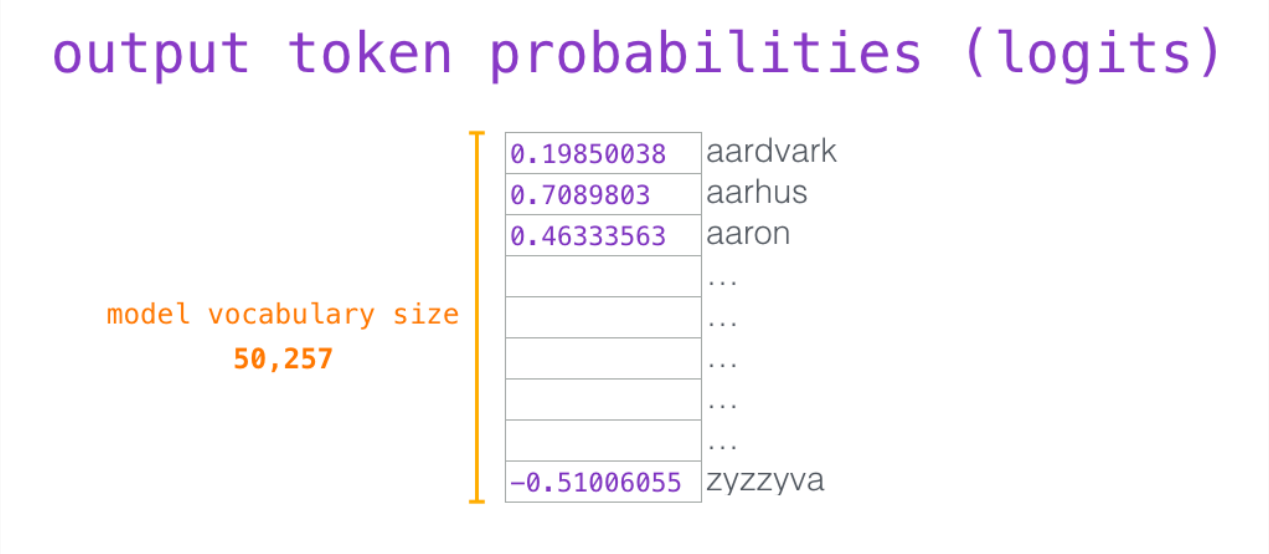


Figure 14: Recall that each row in the embedding matrix corresponds to the embedding of a word in the model's vocabulary. Multiplying the final hidden state by the transposed embedding matrix yields a score for each vocabulary word. **Note:** While we could use a separate weight matrix of size $d \times |V|$ for this projection, using the transposed embedding matrix reduces the number of parameters and promotes consistency between the input and output token spaces.

Output

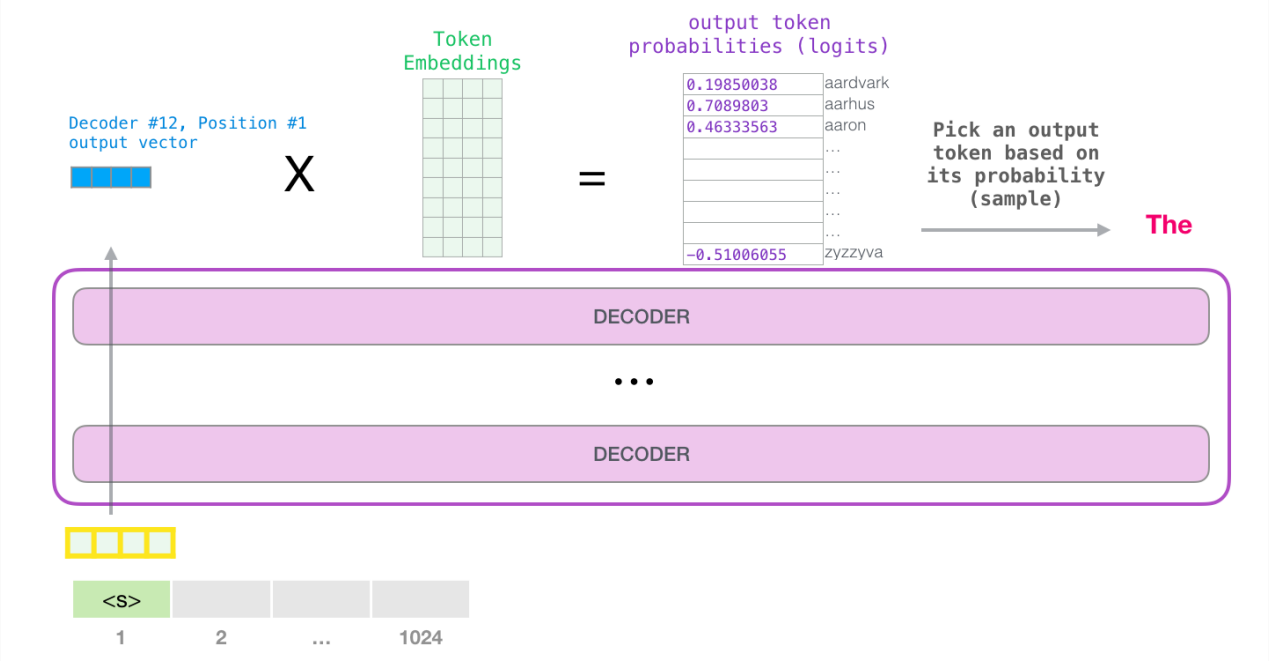


Figure 15: We can then select the token with the highest score (argmax). Alternatively—and often more effectively—we can apply a softmax to convert the logits into probabilities and sample from the resulting distribution.

Autoregressive generation

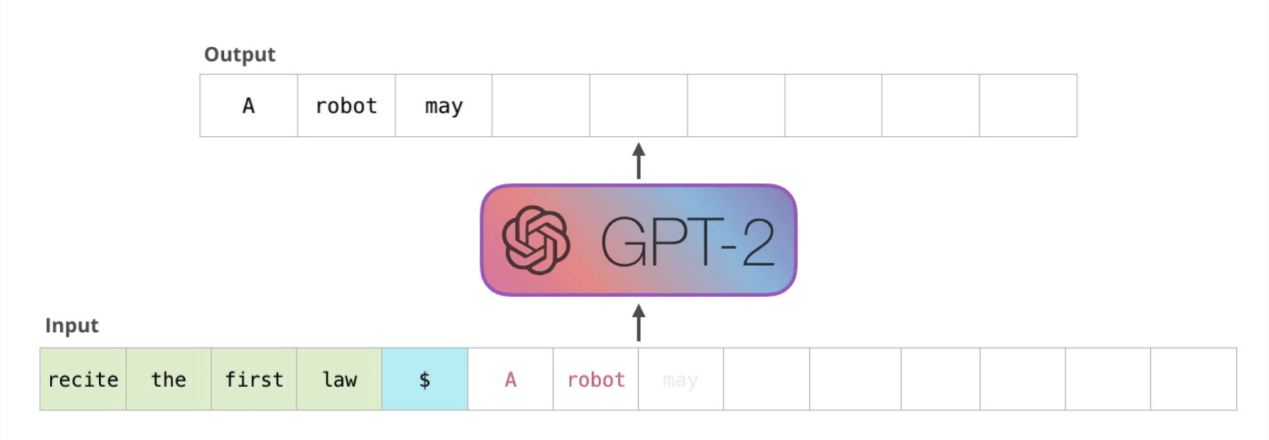


Figure 16: We run a sentence $w_{1:n}$ through the model and use the output corresponding to the last token w_n to predict the next token w_{n+1}. Once w_{n+1} is generated, it is appended to the input sequence, forming $w_{1:n+1}$, which is then used as input for the next step. This process continues autoregressively, generating one token at a time, and stops when the <eos> token is produced or when a maximum sequence length is reached. Animated version here.

BERT vs GPT

BERT

- Uses bidirectional self-attention.
- Trained with masked language modeling (predicting randomly masked tokens).
- Produces contextualized representations based on both left and right context.
- Not designed for language generation tasks.

GPT

- Uses masked (causal) self-attention: each token attends only to previous tokens.
- Trained with next-token prediction (left-to-right).
- Produces representations based on left-side context only.
- Suitable for generative tasks, such as text completion or synthesis.

GPT version...?

Until now, we haven’t discussed any specific version of GPT.

GPT (1)

- Introduced in the paper *Improving Language Understanding by Generative Pre-Training* [Radford et al., 2018].
- Architecture as previously described, 12 layers, 117M parameters.
- Self-supervised pre-training
- Then fine-tuning to solve a variety of different downstream tasks by using a supervised training objective and creating a separate classification head for each task.

GPT 2

- Introduced in the paper *Language Models are Unsupervised Multitask Learners* [Radford et al., 2019].
- A collection of models with up to 1.5 billion parameters.
- Uses the same architecture as GPT, but scaled up with more layers and a larger model dimension (d_{model}).
- Pre-trained on a much larger corpus: WebText, created by scraping diverse web pages.
- No fine-tuning is performed. Instead, tasks are handled via *zero-shot inference*: for example, to classify sentiment, the model is prompted with something like "To which category does the text belong? Positive Sentiment, Negative Sentiment." **No gradient updates are made**.
- GPT-2 does not achieve state-of-the-art performance on most benchmarks, but performance improves consistently with model size—scaling up yields clear gains.

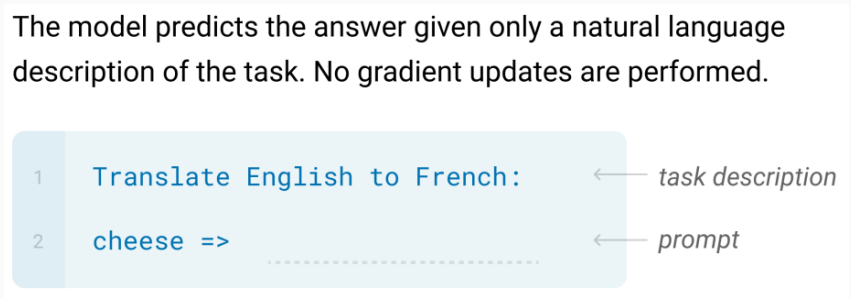


Figure 17: Zero-shot inference.

GPT 3

- Introduced in the paper *Language Models are Few-Shot Learners* [Brown et al., 2020].
- Collection of several models, with the biggest (and standard) one having over 175B parameters (100x larger than the largest GPT model).
- Pretrained on an even bigger dataset called CommonCrawl.
- No fine-tuning. Tasks are handled via *few-shot prompting*: the LLM learns how to perform a task based upon examples that are placed within its context window (the *prompt*). Again, new gradient updates are made.
- This is called *in-context learning*, a new paradigm in NLP. The LLM does not actually "learn"—the model's weights are not updated at all. Rather, the examples in the model's input are used as context for generating a more accurate output.

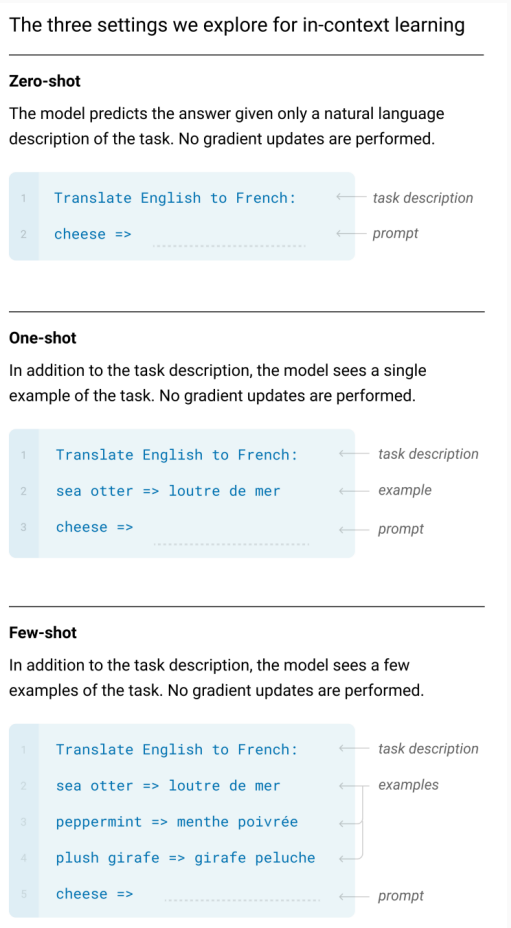


Figure 18: Caption

References i

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. (2020).
Language Models are Few-Shot Learners.
arXiv:2005.14165.

References ii

Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019).
BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.
In Burstein, J., Doran, C., and Solorio, T., editors, *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.

References iii

- Phuong, M. and Hutter, M. (2022).
  Formal Algorithms for Transformers.
  arXiv:2207.09238 [cs].
- Radford, A., Narasimhan, K., Salimans, T., Sutskever, I., et al. (2018).
  Improving language understanding by generative pre-training.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., and Sutskever, I. (2019).
  Language Models are Unsupervised Multitask Learners.

References iv

Rothe, S., Narayan, S., and Severyn, A. (2020). **Leveraging Pre-trained Checkpoints for Sequence Generation Tasks.** *Transactions of the Association for Computational Linguistics*, 8:264–280. arXiv:1907.12461 [cs].

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). **Attention is all you need.** *Advances in neural information processing systems*, 30.